

```
'''
```

Для локальной модели можно обращаться к Ollama через HTTP API:

```
'''typescript
// src/ollama.ts

export async function askLocal(prompt: string): Promise<string> {

  const response = await fetch("http://127.0.0.1:11434/api/generate", {

    method: "POST",

    headers: {

      "Content-Type": "application/json"

    },

    body: JSON.stringify({

      model: "qwen2.5-coder:7b",

      prompt,

      stream: false,

      options: {

        temperature: 0.2,

        num_ctx: 4096

      }

    })

  });

  if (!response.ok) {

    throw new Error(`Ollama HTTP ${response.status}`);

  }

  const data = await response.json() as { response: string };

  return data.response;

}

'''
```

Мост к уже открытой вкладке Gemini через Playwright лучше делать через отдельный профиль Chromium. Playwright подключается к браузеру с включённым remote debugging:

```
'''powershell

$chrome = "${env:ProgramFiles(x86)}\Google\Chrome\Application\chrome.exe"

& $chrome `

  --remote-debugging-port=9222 `

  --user-data-dir="H:\XPE\chrome-profile"

'''
```

В этом окне один раз откройте Gemini и войдите в аккаунт. Затем TypeScript сможет подключиться к вкладке:

```
``typescript
// src/gemini-browser.ts

import { chromium, Page } from "playwright";

export async function connectGemini(): Promise<Page> {

  const browser = await chromium.connectOverCDP(

    "http://127.0.0.1:9222"

  );

  for (const context of browser.contexts()) {

    for (const page of context.pages()) {

      if (page.url().includes("gemini.google.com")) {

        await page.bringToFront();

        return page;

      }

    }

  }

  throw new Error("Вкладка Gemini не найдена");

}

export async function sendToGemini(

  page: Page,

  message: string

): Promise<void> {

  const input = page.locator(

    'textarea, [contenteditable="true"]'

  ).last();

  await input.waitFor({ state: "visible", timeout: 15_000 });

  await input.fill(message);

  await input.press("Enter");

}

export async function readGeminiAnswer(

  page: Page

): Promise<string> {

  await page.waitForTimeout(1500);

  const candidates = page.locator(
```

```

    '[data-message-author-role="model"]', model-response, message-content'
  );

  const count = await candidates.count();

  if (count === 0) {
    return "Ответ Gemini не найден.";
  }

  return (await candidates.last().innerText()).trim();
}

'''

```

Селекторы Gemini могут меняться, поэтому их лучше хранить отдельно в конфигурации:

```

'''typescript
export const selectors = {
  input: [
    "textarea",
    '[contenteditable="true"]'
  ],
  responses: [
    '[data-message-author-role="model"]',
    "model-response",
    "message-content"
  ]
};

'''

```

Центральный оркестратор должен передавать сообщения в едином формате:

```

'''typescript
export type AgentName = "local" | "gemini" | "human";

export type AgentMessage = {
  id: string;
  from: AgentName;
  to: AgentName | "broadcast";
  type: "task" | "proposal" | "review" | "result" | "question";
  content: string;
  files?: string[];
  requiresApproval?: boolean;
};

```

```
timestamp: string;  
};  
'''
```

Пример распределения задачи:

```
'''typescript  
const task = `  
Проект Godot 4.  
Нужно создать систему движения игрока.  
Ограничения:  
- CharacterBody2D;  
- Input Map;  
- WASD;  
- без сторонних библиотек;  
- код должен быть совместим с Godot 4.  
`;  
  
const geminiPrompt = `  
Ты архитектор проекта.  
Проанализируй задачу и предложи структуру решения.  
Не пиши большой объём кода.  
Укажи риски и API Godot 4.  
  
ЗАДАЧА:  
${task}  
`;  
  
const architecture = await sendToGeminiAndRead(geminiPrompt);  
  
const localPrompt = `  
Ты исполнительный программист.  
На основе архитектурного предложения создай точное решение.  
Покажи изменяемые файлы и минимальный патч.  
Не меняй код без необходимости.  
  
ЗАДАЧА:  
${task}  
  
ПРЕДЛОЖЕНИЕ АРХИТЕКТОРА:  
${architecture}  
`;
```

```
const implementation = await askLocal(localPrompt);  
'''
```

Лучший режим для вас — ****полуавтоматический****:

```
'''text
```

1. Вы вводите задачу.
2. Gemini предлагает архитектуру.
3. Локальная модель проверяет предложение.
4. Вы видите диалог в TUI.
5. Вы подтверждаете.
6. Локальная модель меняет файлы.
7. Тесты запускаются автоматически.
8. Ошибка возвращается локальному ИИ и Gemini.

```
'''
```

Добавьте обязательные команды подтверждения:

```
'''text
```

```
/ask gemini <текст>    отправить вопрос Gemini  
/ask local <текст>     отправить вопрос локальной модели  
/debate <текст>        дать задачу обоим агентам  
/approve              разрешить изменение файлов  
/reject              отклонить предложение  
/diff                показать изменения  
/test                запустить тесты  
/stop                остановить текущую задачу
```

```
'''
```

Особенно важен режим `requiresApproval`:

```
'''typescript
```

```
if (message.requiresApproval) {  
    console.log("Изменения ожидают подтверждения пользователя.");  
    console.log("Введите /approve или /reject.");  
}
```

```
'''
```

Не стоит сразу строить полностью автономный цикл. Сначала сделайте минимальный рабочий прототип:

```
'''text
```

TUI → Node.js → Ollama
TUI → Node.js → Playwright → Gemini
Node.js → общий журнал сообщений
'''

Затем добавьте:

'''text
локальный ИИ ↔ Gemini
'''

И только после этого:

'''text
агенты → файлы → Godot → тесты → исправление
'''

Такой подход даст именно то, что вы описали: локальный ИИ будет исполнителем, Gemini — вторым архитектором/ревьюером, а вы останетесь оператором, который может вмешаться в любой момент.

User prompt 22 of 25 - 9/8/2026, 10:59:51 PM:
подожди сначала tui сделаем мне не удобно сообщем еще эти моргания

GPT-5.6 Luna:
<!-- -->
Да, сначала полностью делаем ****TUI****, без Ollama, Gemini и агентов. Мост пока не трогаем.

Основная проблема текущей версии — она постоянно вызывает:

'''powershell
Clear-Host
'''

Из-за этого весь экран заново рисуется и мигает. В новой версии этого не будет.

Сначала сделаем только визуальный прототип в стиле Kinic:

- тёмный фон;
- розово-фиолетовая тема;
- крупный логотип;
- левая боковая панель;

- вкладки;
- центральная область чата;
- поле ввода;
- нижняя строка команд;
- статичные тестовые сообщения;
- навигация клавишами;
- без сетки из лишних рамок;
- без постоянного обновления экрана;
- без подключения моделей.

Технически:

```
'''text
```

экран рисуется один раз

↓

курсор скрывается

↓

изменяются только нужные строки

↓

никакого Clear-Host

↓

никакого мерцания

...

Сделаем сначала такой каркас:

```text

The diagram illustrates the layout of the IDE interface, showing the arrangement of various components. The components are organized into a grid-like structure with labels and dimensions.

- Top Bar:** Contains the XPE logo (100x40), a list of items (1 item, 40x40), and a search icon (40x40).
- Left Panel:** Contains the XPE logo (100x40), a list of items (1 item, 40x40), and a search icon (40x40).
- Main Area:** Contains the XPE logo (100x40), a list of items (1 item, 40x40), and a search icon (40x40).
- Bottom Panel:** Contains the XPE logo (100x40), a list of items (1 item, 40x40), and a search icon (40x40).

The diagram shows the relative positions and sizes of these components, with labels indicating the specific elements and their dimensions.

```
|_____|
| Tab focus Enter send /new Esc close Q quit |
|_____|
```

'''

Сначала добиваемся нормального визуального вида и отсутствия мигания. Только после вашего одобрения добавим ввод текста, переключение панелей и реальные данные.

-----

User prompt 23 of 25 - 9/8/2026, 11:00:53 PM:

почему нельзя блин как в linux сделать красиво и со вкусом

GPT-5.6 Luna:

<!-- -->

Можно. Проблема не в PowerShell 7, а в том, что предыдущий вариант был сделан на примитивном 'Write-Host' и ручных рамках. Это не настоящий TUI, поэтому он и выглядел плохо.

Для интерфейса уровня Linux-программ нужно использовать TUI-фреймворк, а не рисовать весь экран вручную. Оптимальный вариант для вас:

'''text

PowerShell 7

↓ запускает

Node.js + TypeScript

↓

Ink / React или blessed

↓

настоящий TUI без мерцания

'''

Но для сложного интерфейса с панелями, вкладками, фокусом, прокруткой и чатами лучше использовать **Python Textual** или **Go Bubble Tea**. На Windows они тоже работают в PowerShell 7 и выглядят как нормальные Linux TUI.

Я бы выбрал:

'''text

Python + Textual

'''

Почему:

- нормальные панели и виджеты;
- CSS-подобное оформление;

- фиксированные области;
- прокрутка;
- поля ввода;
- вкладки;
- фокус клавиатуры;
- отсутствие мерцания;
- простое создание интерфейса в стиле Kinic;
- легко подключить Ollama и Node.js позже.

Пример структуры:

```
'''text
xpe/
├─ tui/
│ └─ app.py
│ └─ styles.tcss
│ └─ widgets/
├─ bridge/
│ └─ Node.js
└─ workspace/
'''
```

Установка:

```
'''powershell
py -m pip install textual
'''
```

Минимальный настоящий интерфейс будет выглядеть примерно так:

```
'''python
from textual.app import App, ComposeResult
from textual.containers import Horizontal, Vertical
from textual.widgets import Header, Footer, Input, Label, ListItem, ListView, Static, TabbedContent, TabPane

class XPE(App):

 CSS_PATH = "styles.tcss"

 def compose(self) -> ComposeResult:
 yield Header(show_clock=False)

 with Horizontal(id="layout"):
```

```

with Vertical(id="sidebar"):

 yield Label("K9 / XPE", id="logo")

 yield Label("Tabs", classes="section")

 yield Label("Memories", classes="tab active")

 yield Label("Agents", classes="tab")

 yield Label("Sessions", classes="tab")

 yield Label("Search", classes="section")

 yield ListView(

 ListItem(Label("● CODER")),

 ListItem(Label("○ DEBUGGER")),

 ListItem(Label("○ REVIEWER")),

 id="agents"

)

with Vertical(id="main"):

 yield Label("◆ Chat [XPE SESSION]", id="title")

 yield Static(

 "You:\n"

 "Нужно создать систему движения игрока.\n\n"

 "Local Coder:\n"

 "Ожидаю задачу...",

 id="chat"

)

 yield Input(

 placeholder="Ask about this item... (Enter to send)",

 id="input"

)

yield Footer()

if __name__ == "__main__":

 XPE().run()

'''

Стили:

'''css

Screen {

 background: #18181b;

 color: #d4d4d8;

}

```

```
Header {

 background: #18181b;

 color: #f472b6;

 dock: top;

}
```

```
Footer {

 background: #18181b;

 color: #71717a;

}
```

```
#layout {

 height: 1fr;

}
```

```
#sidebar {

 width: 28;

 border: solid #d9469f;

 padding: 1 2;

}
```

```
#main {

 width: 1fr;

 border: solid #d9469f;

 padding: 1 2;

}
```

```
#logo {

 color: #f472b6;

 text-style: bold;

 margin-bottom: 2;

}
```

```
.section {

 color: #a855f7;

 margin-top: 1;

}
```

```
.tab {

 color: #a1a1aa;

 padding: 0 1;
```

```

}

.active {
 color: #f472b6;
 text-style: bold;
}

#agents {
 background: #18181b;
 border: none;
 height: auto;
}

#title {
 color: #f472b6;
 text-style: bold;
}

#chat {
 height: 1fr;
 border: solid #3f3f46;
 padding: 1 2;
 margin: 1 0;
}

#input {
 border: solid #d9469f;
 background: #202023;
}
'''

```

Запуск из PowerShell:

```

'''powershell
cd H:\XPE\tui
py .\app.py
'''

```

Это уже будет ближе к Linux TUI, а не к мигающему PowerShell-скрипту.

Дальше можно сделать крупный ASCII-логотип `K9`, вкладки `Memories / Insert / Create / Market / Settings`, нормальную историю чата, навигацию `Tab`, горячие клавиши и подключение к Ollama. Сначала нужно перейти на нормальный TUI-фреймворк — вручную рисовать такой интерфейс через PowerShell не стоит.

-----

User prompt 24 of 25 - 9/8/2026, 11:02:14 PM:

```
PS C:\Users\RJ> cd H:\XPE\Models\qwen-uncensored
```

```
PS H:\XPE\Models\qwen-uncensored> cd ../..
```

```
PS H:\XPE> py -m pip install textual
```

Can't find a default Python.

```
PS H:\XPE> cd C:\Python314
```

```
PS C:\Python314> py -m pip install textual
```

Can't find a default Python.

```
PS C:\Python314>
```

GPT-5.6 Luna:

<!-- \*\*Explaining Python launcher issues\*\*

I need to explain the issue with the Python launcher being absent or not configured, which might relate to the Python executable itself. I should check the version using 'python --version' or look for the command with 'Get-Command python' and pip. It's possible that the Python314 folder exists without 'python.exe'. In that case, I could use 'C:\Python314\python.exe -m pip...'. Additionally, if pip is absent, I might need to run '-m ensurepip'. Lastly, the error "Can't find default Python" likely stems from no registered install. -->

'py' у вас установлен или вызывается, но не видит зарегистрированный Python. Переход в папку 'C:\Python314' не меняет настройки запуска.

Сначала проверьте, есть ли сам интерпретатор:

```
'''powershell
```

```
Test-Path C:\Python314\python.exe
```

```
'''
```

Если результат:

```
'''text
```

```
True
```

```
'''
```

используйте прямой путь:

```
'''powershell
```

```
C:\Python314\python.exe -m pip install textual
```

```
'''
```

Если 'pip' отсутствует:

```
```powershell
C:\Python314\python.exe -m ensurepip --upgrade
C:\Python314\python.exe -m pip install --upgrade pip
C:\Python314\python.exe -m pip install textual
```
```

Проверьте Python:

```
```powershell
C:\Python314\python.exe --version
```
```

Если получите ошибку, найдите настоящий путь:

```
```powershell
Get-Command python -ErrorAction SilentlyContinue
Get-Command python3 -ErrorAction SilentlyContinue
Get-Command py -ErrorAction SilentlyContinue
```
```

Также:

```
```powershell
where.exe python
where.exe py
```
```

Если Python установлен в другой папке, команда будет выглядеть так:

```
```powershell
"C:\ПУТЬ\К\python.exe" -m pip install textual
```
```

Можно проверить стандартные варианты:

```
```powershell
Get-ChildItem `
    "$env:LocalAppData\Programs\Python" `
    "C:\Program Files\Python*" `
    "C:\Python*" `
    -Filter python.exe `
```
```

```
-Recurse `
-ErrorAction SilentlyContinue
'''
```

После установки Textual запускать приложение нужно также через конкретный Python:

```
```powershell
C:\Python314\python.exe H:\XPE\tui\app.py
'''
```

Но сначала выполните и пришлите результат двух команд:

```
```powershell
Test-Path C:\Python314\python.exe
Get-Command python -ErrorAction SilentlyContinue
'''
```

С высокой вероятностью нужно будет использовать 'C:\Python314\python.exe', а не 'py'.

-----

User prompt 25 of 25 - 9/8/2026, 11:03:02 PM:

PowerShell 7.6.0

A new PowerShell stable release is available: v7.6.5

Upgrade now, or check out the release page at:

<https://aka.ms/PowerShell-Release?tag=v7.6.5>

PS C:\Users\RJ> cd H:\XPE\Models\qwen-uncensored

PS H:\XPE\Models\qwen-uncensored> cd ../..

PS H:\XPE> py -m pip install textual

Can't find a default Python.

PS H:\XPE> cd C:\Python314

PS C:\Python314> py -m pip install textual

Can't find a default Python.

PS C:\Python314> Test-Path C:\Python314\python.exe

True

PS C:\Python314> C:\Python314\python.exe -m pip install textual

Defaulting to user installation because normal site-packages is not writeable

Collecting textual

Downloading textual-8.2.8-py3-none-any.whl.metadata (9.1 kB)

Collecting markdown-it-py>=2.1.0 (from markdown-it-py[linkify]>=2.1.0->textual)